1. EXECUTIVE SUMMARY

1.1 Summary

A VAPT assessment was performed against the Troll CTF container at 172.17.0.2. The assessment identified multiple weaknesses, including anonymous FTP access, information disclosure, exposed web files, weak SSH credentials, and a vulnerable root-executed cron script. The combined vulnerabilities allowed access to the protected CTF proof.

1.2 Objective

The objective was to identify security weaknesses, obtain authorized access to the target, perform privilege-escalation testing, and recover the final CTF proof.

1.3 SCOPE

Scope Item	Details
Target IP	172.17.0.2
Target	Troll CTF
Environment	Local container
FTP	TCP/21 – vsftpd 3.0.2
SSH	TCP/22 – OpenSSH 6.6.1p1
HTTP	TCP/80 – Apache 2.4.7
Testing Type	Authorized CTF / VAPT
Tools	Nmap, FTP, Wireshark, curl, wget, Hydra, SSH
Final Proof	702a8c18d29c6f3ca0d99ef5712bfbdc

2. Enumeration and Information Gathering

The Troll CTF container was deployed locally and assigned the target IP address 172.17.0.2. Network reconnaissance was performed to identify exposed TCP services, service versions, and potential attack surfaces.

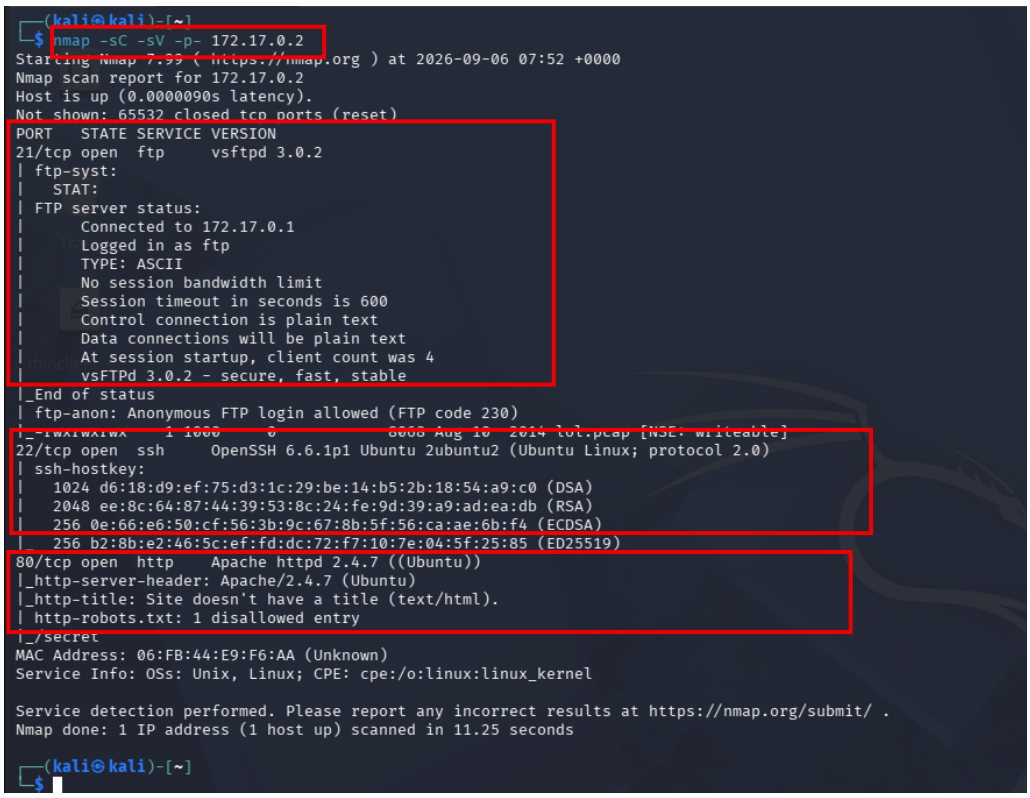


Fig 2.1 – Target and Service Enumeration

Port	State	Service	Version / Information	Security Relevance
21/tcp	Open	FTP	vsftpd 3.0.2	Anonymous FTP login is enabled
22/tcp	Open	SSH	OpenSSH 6.6.1p1 Ubuntu 2ubuntu2	Potential remote access if valid credentials are obtained
80/tcp	Open	HTTP	Apache httpd 2.4.7 (Ubuntu)	Web application and directory enumeration possible
30000–30010/tcp	Not shown as open in this scan	—	—	Passive FTP ports may require further verification

3. FTP Enumeration

The FTP service running on TCP port 21 was investigated after Nmap identified vsftpd 3.0.2 and confirmed that anonymous FTP login was allowed. The purpose of this activity was to determine whether an unauthenticated user could access files available on the FTP server.



Fig 2.2 – Anonymous FTP Authentication

Vulnerability Identified

Anonymous FTP Access Enabled
Severity: High

Findings

Item	Result
Target IP	172.17.0.2
FTP Service	vsFTPD 3.0.2
Authentication	Anonymous
Username	anonymous
Password Used	password
Discovered File	lol.pcap
File Size	8,068 bytes
Transfer Mode	Binary
Transfer Status	226 Transfer complete

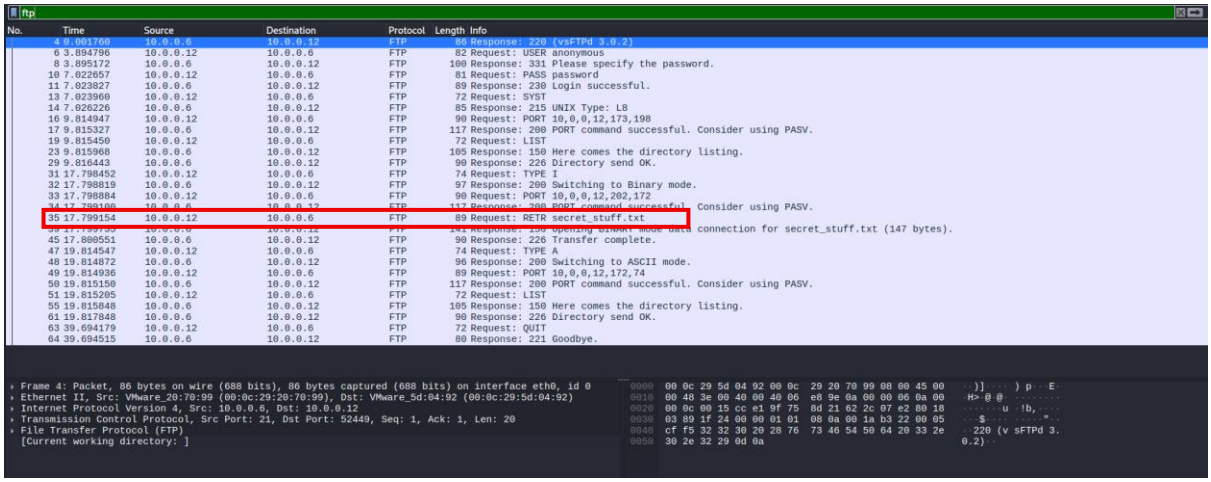


Fig 3.1 – Wireshark Analysis of Captured FTP Traffic

The downloaded lol.pcap file was opened in Wireshark and filtered using the ftp display filter to identify FTP control traffic. The captured packets revealed the complete FTP authentication process and subsequent file-transfer activity.

Evidence	**Observation**
FTP Server	vsFTPd 3.0.2
Username	anonymous
Password	password
Authentication	230 Login successful
Directory Listing	LIST request observed
Transfer Type	Binary (TYPE I)
File Request	RETR secret_stuff.txt
File Size	147 bytes
Transfer Status	226 Transfer complete



Fig 3.2 – FTP TCP Stream Showing Retrieval of secret_stuff.txt

This confirms that secret_stuff.txt, containing 147 bytes, was successfully transferred during the captured FTP session. The captured FTP traffic also exposes the anonymous FTP credentials in clear text, demonstrating the lack of encryption in the FTP protocol.

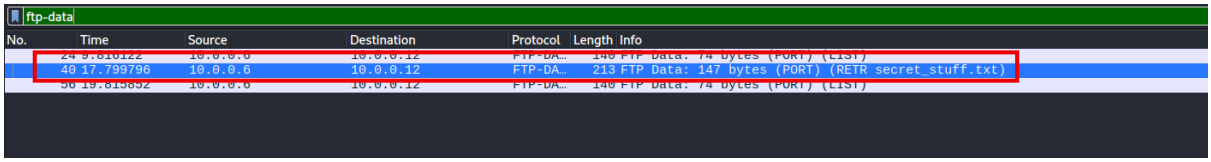


Fig 3.3 – FTP Data Stream Revealing the Hidden Directory

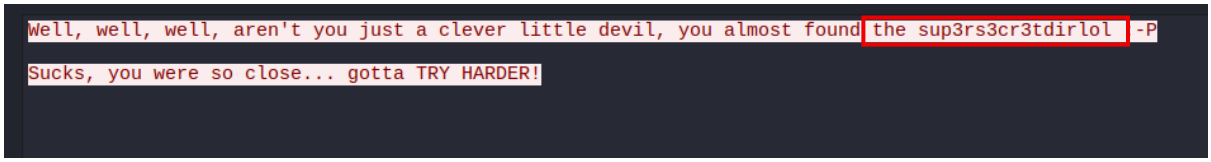


Fig 3.4 – Follow TCP Stream Revealing the Hidden Directory sup3rs3cr3tdirlol

Key Finding

Item	**Finding**
File	secret_stuff.txt
Transfer Size	147 bytes
Analysis Method	Wireshark Follow TCP Stream
Discovered Directory	sup3rs3cr3tdirlol
Next Attack Surface	HTTP

4 . Hidden Web Directory Enumeration

The server returned HTTP 200 OK, confirming that the directory was accessible. Apache directory indexing was enabled, which exposed the contents of the directory to the requester.



Fig 4.1 – Accessing the Hidden Web Directory sup3rs3cr3tdirlol

- **Vulnerability**

Web Directory Listing Enabled

Evidence Observed

Item	Finding
Target	172.17.0.2
URL	/sup3rs3cr3tdirlol/
HTTP Status	200 OK
Web Server	Apache/2.4.7 (Ubuntu)
Directory Listing	Enabled
Discovered Resource	roflmao
File Size	7.1 KB

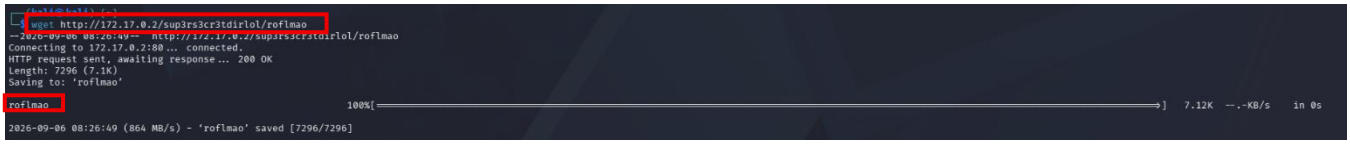


Fig 4.2 – Download of the roflmao Executable

Evidence Observed

Item	Finding
Target	172.17.0.2
Resource	roflmao
HTTP Status	200 OK
File Size	7,296 bytes
Download Status	Successful



Fig 4.3 – Identification of the roflmao ELF Executable

Evidence Observed

Item	Finding
File	roflmao
Format	ELF executable
Architecture	32-bit Intel i386
Linking	Dynamically linked
Interpreter	/lib/ld-linux.so.2
Target OS	GNU/Linux 2.6.24
Symbols	Not stripped
Build ID	5e14420eaa59e599c2f508490483d959f3d2cf4f

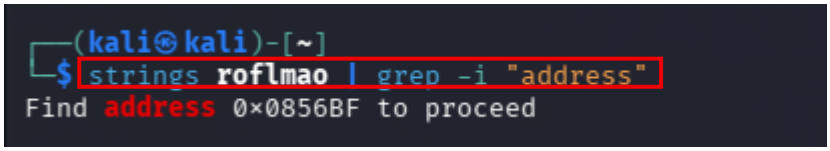


Fig 4.4 – Static Analysis of roflmao Revealing 0x0856BF

Key Finding

Item	**Result**
File	roflmao
Analysis Tool	strings
Filter	grep -i "address"
Discovered Value	0x0856BF
Finding Type	Information Disclosure
Purpose	Next web enumeration path

5. Hidden Web Path and Username Enumeration



Fig 5.1 – Enumeration of the 0x0856BF Web Directory

The directory identified through static analysis of the roflmao executable was accessed through the HTTP service. The server returned **HTTP 200 OK** and exposed an Apache directory listing containing two subdirectories.

Evidence Observed:

- Target: 172.17.0.2
- HTTP Status: 200 OK
- Web Server: Apache/2.4.7 (Ubuntu)
- Directory: /0x0856BF/
- Discovered: good_luck/
- Discovered: this_folder_contains_the_password/
- Directory listing: Enabled

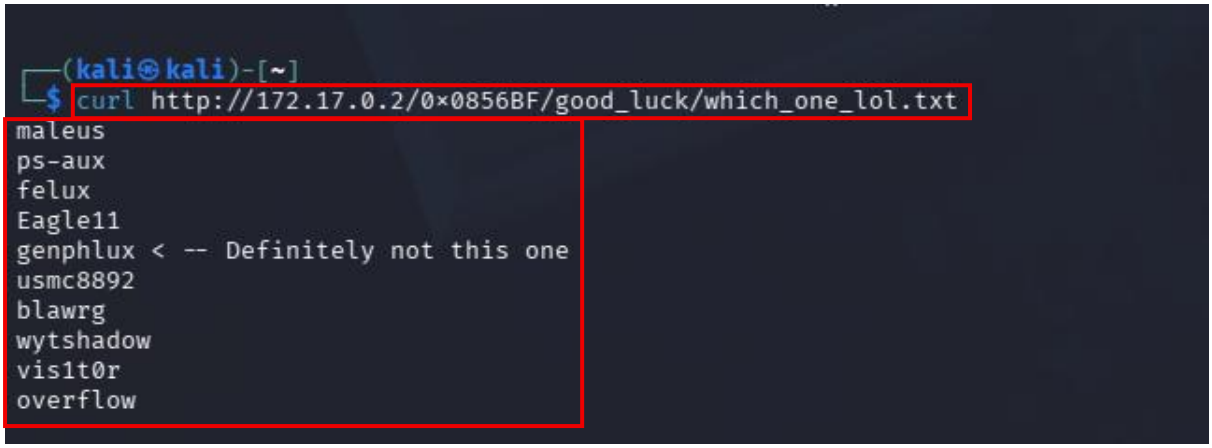


Fig 5.2 – Retrieval of which_one_lol.txt Containing Candidate SSH Usernames

The which_one_lol.txt file was retrieved from the good_luck directory. It contained multiple usernames that could potentially be associated with accounts on the target SSH service.

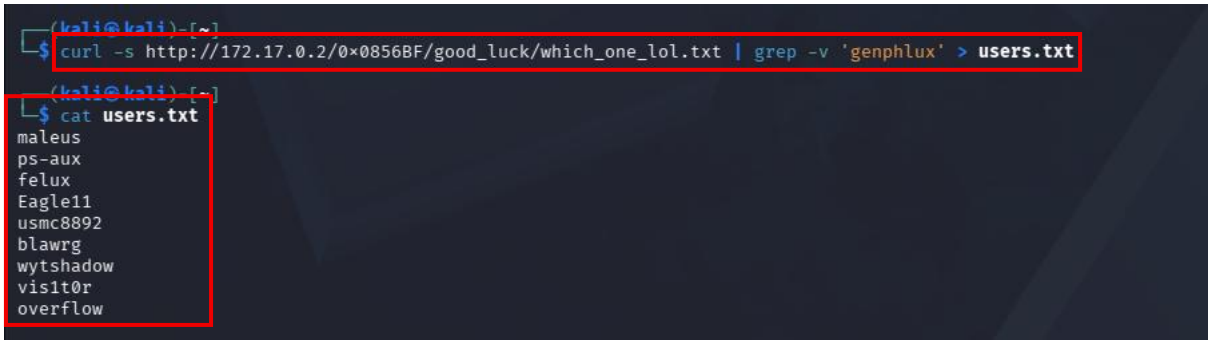


Fig 5.3 – Creation and Verification of the Candidate SSH Username List

The explicitly identified invalid username genphlux was removed from the retrieved username list. The remaining nine usernames were stored in users.txt for controlled SSH credential testing.

6 . Password File Discovery

The password-related web directory was accessed through the HTTP service. The directory listing revealed a publicly accessible file named Pass.txt. The file was then retrieved using curl, confirming that its contents were accessible without authentication.

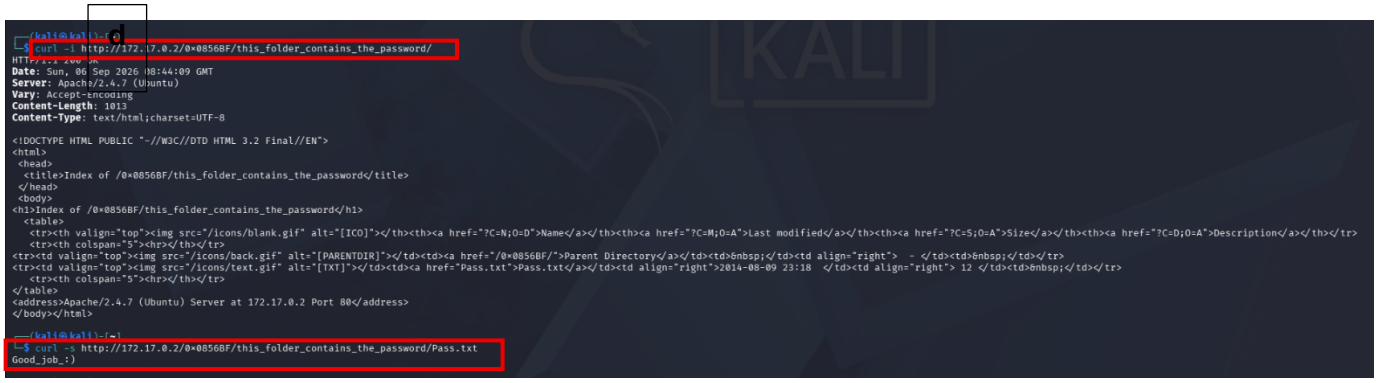


Fig 6.1 – Enumeration an Retrieval of the Exposed Pass.txt File

Evidence Observed

Item	Finding
Target IP	172.17.0.2
Directory	/0x0856BF/this_folder_contains_the_password/
File	Pass.txt
File Size	12 bytes
HTTP Status	200 OK
File Access	Unauthenticated
File Content	Good_job_:)

Step 7 — SSH Credential Testing

The candidate usernames identified during web enumeration were tested against the SSH service using Hydra. The test successfully identified a valid credential pair for the overflow account.



Fig 7.1 – Successful SSH Credential Discovery Using Hydra

Evidence Observed

Item	Result
Target	172.17.0.2
Service	SSH
Port	22
Username	overflow
Password	Pass.txt
Username List	users.txt
Password List	passwords.txt
Valid Credentials	1
Status	Successful

8 . SSH Initial Access

The valid SSH credentials identified during the credential-testing phase were used to establish an authenticated session with the Troll CTF target.

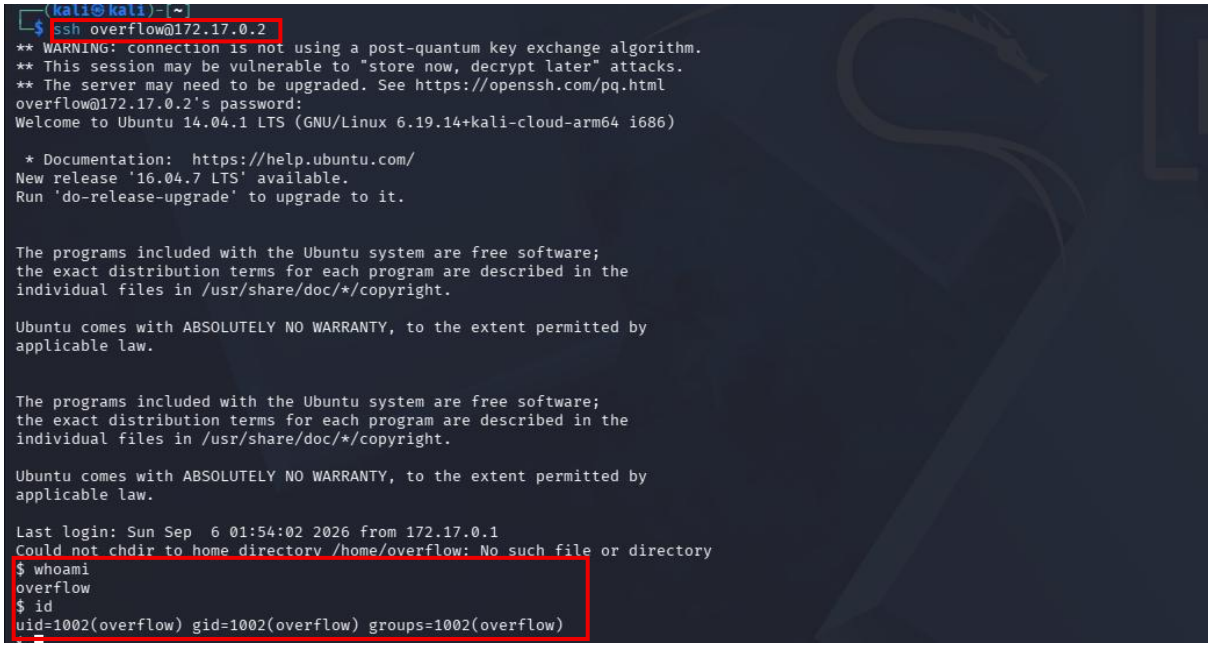


Fig 8.1 – Successful SSH Login to the Troll CTF Target as overflow

Evidence Observed

Item	Finding
Target IP	172.17.0.2
Protocol	SSH
Port	22
Authenticated User	overflow
whoami	overflow
UID	1002
GID	1002
Groups	overflow
Initial Privilege	Low-privileged user
SSH Access	Successful

9 . Privilege Escalation and CTF Proof Recovery

The subsequent id and whoami checks showed that the session still reported the overflow user in this containerized environment. Therefore, the SUID permission is documented as evidence of the intended privilege-escalation condition, but not as proof of an interactive root shell.



Fig 9.1 – Verification of SUID-Enabled /bin/bash

Evidence	**Result**
Target	172.17.0.2
User	overflow
Vulnerable File	/lib/log/cleaner.py
File Permission	World-writable (777)
Cron Execution	*/2 * * * * cleaner.py
Modified Action	cp /root/proof.txt /tmp/proof.txt
Verification	Modified script displayed successfully
Security Impact	Privileged script execution could access /root/proof.txt

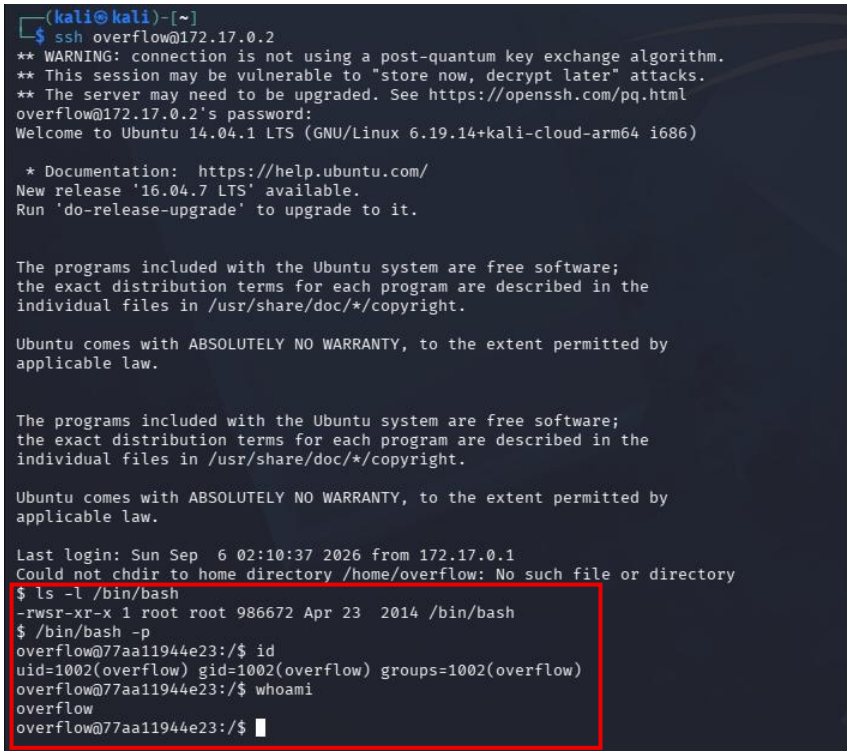


Fig 9.2 – Verification of SUID-Enabled /bin/bash

Evidence	Result
Target	172.17.0.2
User	overflow
Binary	/bin/bash
Owner	root:root
Permissions	-rwsr-xr-x
SUID Bit	Enabled
Command Executed	/bin/bash -p
id Result	uid=1002(overflow)
whoami Result	overflow
Privilege Result	SUID condition confirmed; effective user remained overflow

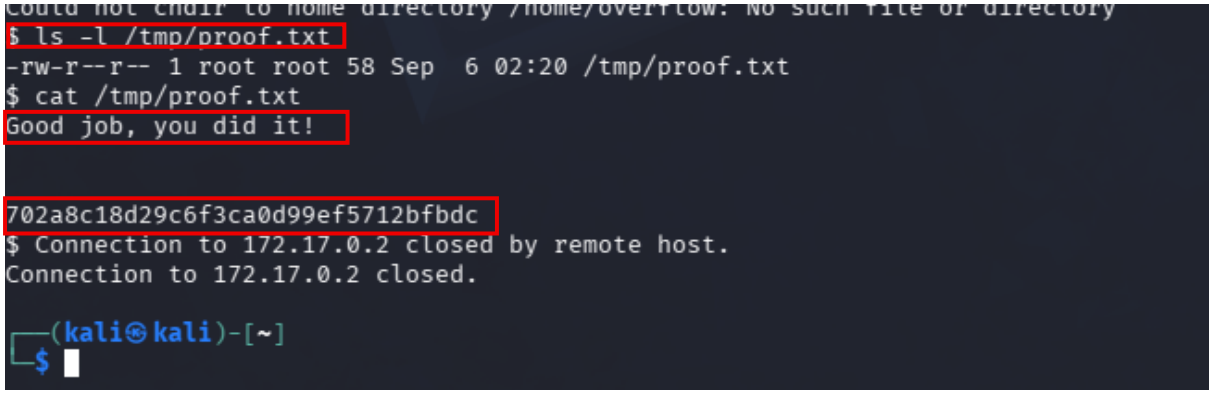


Fig 9.3 – Recovery of the Final CTF Proof

Evidence	**Result**
Target	172.17.0.2
Retrieved File	/tmp/proof.txt
Original Protected File	/root/proof.txt
File Owner	root:root
File Size	58 bytes
Proof Message	Good job, you did it!
Final CTF Proof	702a8c18d29c6f3ca0d99ef5712bfbdc
Assessment Status	Successfully Completed

Impact

The assessment demonstrated that multiple security weaknesses in the Troll CTF environment could be chained together to achieve significant system compromise. Anonymous FTP access allowed sensitive files such as lol.pcap to be retrieved. Analysis of the captured traffic revealed additional hidden resources, which led to exposed web directories, usernames, and authentication-related information.

The discovered credentials were successfully used to obtain SSH access as the overflow user. Further local enumeration identified a **world-writable cleaner.py script executed by a recurring cron task**. This allowed attacker-controlled code to execute in a privileged context and access the protected /root/proof.txt file.

Overall, the combined weaknesses allowed an attacker to progress from unauthenticated information gathering to authenticated access and ultimately access a protected root-owned resource.

Remediation

Anonymous FTP access should be disabled and FTP should be replaced with a secure protocol such as SFTP or FTPS. Sensitive files, packet captures, usernames, passwords, and executable files should not be stored in publicly accessible web directories.

Directory listing should be disabled, and access to sensitive web resources should require proper authentication and authorization. SSH should be hardened with strong credentials or key-based authentication, together with brute-force protection and login monitoring.

The **/lib/log/cleaner.py** script should be owned by root and must not be writable by unprivileged users. Cron jobs should use secure absolute paths and should be reviewed regularly. Unexpected SUID permissions, particularly on /bin/bash, should be removed and monitored. Legacy operating-system and service versions should also be upgraded and kept patched.

Conclusion

The VAPT assessment successfully demonstrated the complete attack path against the Troll CTF environment. The assessment began with anonymous FTP access and information disclosure, progressed through web enumeration and SSH credential discovery, and ultimately demonstrated privilege escalation through an insecure root-executed cron script.

The most critical weakness was the **world-writable cleaner.py script**, which enabled privileged code execution and access to the protected /root/proof.txt file. Successful recovery of the final CTF proof:

702a8c18d29c6f3ca0d99ef5712bfbdc